SYCL-Based Bitonic Sorting for Parallel Architectures

Jurica Ćapin
Department of Computer Science
University of Salerno
Italy

***Abstract*—This paper presents a parallel implementation of the Bitonic Sorting algorithm using SYCL, a cross-platform abstraction for heterogeneous computing. Four optimized versions of the algorithm are developed: a basic version, a version with local memory usage, a version using subgroups, and a version employing tiling. We evaluate their performance across multiple configurations and provide a visual and runtime comparison. The results demonstrate significant performance improvements with each optimization, particularly with subgroup and tiling techniques.**

I. Introduction

Motivation. Bitonic sorting is a parallel sorting algorithm especially suited for hardware with SIMD or GPU capabilities. With the growing need for platform-independent parallel code, SYCL offers a promising approach to harness heterogeneous hardware capabilities. This paper aims to explore SYCL's suitability for parallel sorting through multiple optimized versions of bitonic sort.

Related work. Numerous works have explored GPU-based bitonic sort implementations using CUDA or OpenCL. However, fewer studies have demonstrated its efficiency on SYCL, a Khronos standard that enables portability across CPU, GPU, and FPGA. This work bridges that gap and explores performance trade-offs across different optimization strategies using SYCL.

II. Background

Bitonic sort is a comparison-based parallel algorithm that operates in $\log^2(n)$ phases for sorting n elements. It creates a bitonic sequence and then repeatedly merges it. Due to its highly structured data-access pattern, bitonic sort is well-suited for parallel processing.

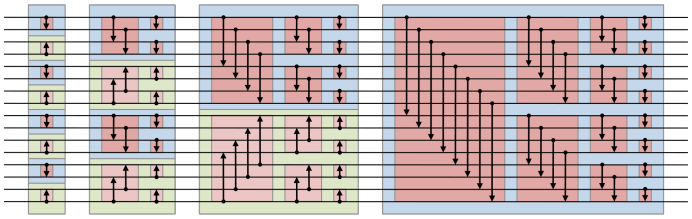

Fig. 1. Bitonic sort visualization for array of 16 elements

SYCL is a high-level programming model that builds on OpenCL and provides modern C++ abstractions for heterogeneous computing. Key SYCL features include unified memory access, hierarchical parallelism, subgroups, and local memory, which we exploit in our implementations.

Sorting algorithms.

Bitonic Sort[2] is a *comparison-based*, *parallel-friendly* sorting algorithm designed for sequences of length $n = 2^k$. It is especially well-suited for parallel architectures (e.g., GPUs, FPGAs) due to its regular control flow and predictable memory access patterns.

a) Core Idea.: Bitonic Sort relies on the concept of a *bitonic sequence*: a sequence that is first monotonically increasing and then decreasing, or vice versa. For example, the sequence

$$[2, 4, 6, 8, 7, 5, 3, 1]$$

is bitonic.

The sorting process has two major stages:

1) **Bitonic Sequence Construction:** Recursively split the array and sort halves in alternating directions (ascending, descending) to form bitonic sequences.
2) **Bitonic Merge:** Recursively compare and conditionally swap elements to convert each bitonic sequence into a fully sorted (monotonic) sequence.

b) High-Level Algorithm.: Let A be the input array of size $n = 2^k$. Bitonic Sort is defined recursively as follows:

1) Sort the first half of A in ascending order.
2) Sort the second half of A in descending order.
3) Merge the entire sequence using Bitonic Merge.

The **Bitonic Merge** step performs the following:

- For each index $i \in [0, n/2)$, compare $A[i]$ with $A[i + n/2]$. Swap if they are out of order.
- Recursively apply Bitonic Merge to the first and second halves.

c) Iterative Version.: For parallel hardware, the recursive formulation is transformed into an iterative version using nested loops:

```
for (size_t k = 2; k <= N; k *= 2) {
  for (size_t j = k / 2; j > 0; j /= 2) {
    // Compare and conditionally swap elements at
  }
}
```

Each iteration performs comparisons between indices i and $i \oplus j$, where $\oplus$ is the bitwise XOR. The sorting direction is determined by whether $i \& k = 0$.

d) Complexity.:

- **Time Complexity:**
  - Sequential: $O(n \log^2 n)$
  - Parallel (ideal case): $O(\log^2 n)$
- **Space Complexity:** $O(n)$
- **Parallelism:** High; comparisons in each stage are independent.

e) Advantages.:

- Deterministic and data-independent execution path.
- Regular memory access patterns—ideal for GPUs and vector processors.
- Highly parallelizable and scalable.

f) Limitations.:

- Requires input size to be a power of two (padding may be needed).
- Less efficient in sequential execution compared to $O(n \log n)$ algorithms like MergeSort or QuickSort.

III. Your Proposed Method

We implement and compare four versions of the bitonic sorting algorithm:

Version 1 (Basic): A straightforward SYCL implementation using global memory only.

```
void bitonic_sort_v1(std::vector<int>& data) {
  const size_t N = data.size();
  if (!is_power_of_two(N)) {
    std::cerr << "Array size must be a power of
    two.\n";
    return;
  }

  // Create a SYCL queue for device execution
  queue q;

  // Create a buffer to manage data on the
    device
  buffer<int, 1> data_buf(data.data(),
    range<1>(N));

  // Outer loop for the size of the bitonic
    sequence
  for (size_t k = 2; k <= N; k *= 2) {
    // Inner loop for the size of the
    subsequences
    for (size_t j = k / 2; j > 0; j /= 2) {
      // Submit a command group to the queue
      q.submit([&](handler& h) {
        // Get access to the buffer
        auto d =
    data_buf.get_access<access::mode::read_write>(h);
        // Launch a parallel kernel
        h.parallel_for(range<1>(N), [=](id<1>
    i) {
          // Compute the index to compare with
          size_t ixj = i[0] ^ j;
          // Ensure the index is valid
          if (ixj > i[0]) {
            // Determine sorting direction
            bool ascending = ((i[0] & k) == 0);
            // Compare and swap if needed
            if ((d[i[0]] > d[ixj]) ==
    ascending) {
              int temp = d[i[0]];
              d[i[0]] = d[ixj];
              d[ixj] = temp;
            }
          }
        });
      });
    }
  }
  // Wait for all tasks in the queue to complete
  q.wait();
}
```

Listing 1. Bitonic Sort V1

Version 2 (Local Memory): Optimizes memory access by staging elements in local memory, reducing global memory overhead.

```
void bitonic_sort_v2(std::vector<int>& data) {
  // Create a SYCL queue for device execution
  queue q;
  // Create a buffer to manage data on the
    device
  buffer<int, 1> data_buf(data.data(),
    range<1>(N));

  // Outer loop for the size of the bitonic
    sequence
  for (size_t k = 2; k <= N; k *= 2) {
    // Inner loop for the size of the
    subsequences
    for (size_t j = k / 2; j > 0; j /= 2) {
      // Submit a command group to the queue
      q.submit([&](handler& h) {
        // Get access to the buffer
        auto d =
    data_buf.get_access<access::mode::read_write>(h);
        // Create a local memory accessor
        accessor<int, 1,
    access::mode::read_write,
    access::target::local>
    local_mem(range<1>(N), h);
        // Launch a parallel kernel
        h.parallel_for(nd_range<1>(range<1>(N),
    range<1>(N)), [=](nd_item<1> item) {
          // Get the global ID of the current
    thread
          size_t i = item.get_global_id(0);
          // Compute the index to compare with
          size_t ixj = i ^ j;
          // Load data into local memory
          // Copy data from global memory to
    local memory
          local_mem[i] = d[i];
          // Synchronize threads in the
    workgroup

    item.barrier(access::fence_space::local_space);
          // Ensure the index is valid
          if (ixj > i) {
            // Determine sorting direction
            bool ascending = ((i & k) == 0);
            // Compare and swap if needed
            if ((local_mem[i] > local_mem[ixj])
    == ascending) {
              // Temporary variable for swapping
              int temp = local_mem[i];
              // Swap elements in local memory
              local_mem[i] = local_mem[ixj];
              // Complete the swap
              local_mem[ixj] = temp;
            }
          }
          // Synchronize threads in the
    workgroup

    item.barrier(access::fence_space::local_space);
          // Write back to global memory
          // Copy sorted data from local memory
    to global memory
          d[i] = local_mem[i];
        });
      });
    }
  }
  q.wait();
}
```

Listing 2. Bitonic Sort V2

Version 3 (Subgroups): Uses SYCL subgroups to perform intra-group communication and exchange, allowing fine-grained parallelism.

```
void bitonic_sort_v3(std::vector<int>& data) {
  // Create a SYCL queue for device execution
  queue q;
  // Create a buffer to manage data on the
    device
  buffer<int, 1> data_buf(data.data(),
    range<1>(N));

  // Outer loop for the size of the bitonic
    sequence
  for (size_t k = 2; k <= N; k *= 2) {
    // Inner loop for the size of the
    subsequences
    for (size_t j = k / 2; j > 0; j /= 2) {
      // Submit a command group to the queue
      q.submit([&](handler& h) {
        // Get access to the buffer
        auto d =
    data_buf.get_access<access::mode::read_write>(h);
        // Launch a parallel kernel with
    subgroup optimization
        h.parallel_for(nd_range<1>(range<1>(N),
    range<1>(16)), [=](nd_item<1> item) {
          // Get the subgroup for the current
    thread
          auto sg = item.get_sub_group();
          // Get the global ID of the current
    thread
          size_t i = item.get_global_id(0);
          // Compute the index to compare with
          size_t ixj = i ^ j;
          // Load the current element
          int val = d[i];
          // Load the partner element
          int partner_val = d[ixj];
          // Perform subgroup exchange
          if (ixj > i) {
            // Determine sorting direction
            bool ascending = ((i & k) == 0);
            // Compute the minimum value
            int min_val = sycl::min(val,
    partner_val);
            // Compute the maximum value
            int max_val = sycl::max(val,
    partner_val);
            if (ascending) {
              // Assign the minimum value to
    the current element
              val = min_val;
              // Assign the maximum value to
    the partner element
              partner_val = max_val;
            } else {
              val = max_val;
              partner_val = min_val;
            }
          }
          // Write back to global memory
          d[i] = val;
          d[ixj] = partner_val;
        });
      });
    }
  }
  q.wait();
}
```

Listing 3. Bitonic Sort V3

Version 4 (Tiling): Divides the input array into tiles for better memory access patterns, with parallelism exploited within each tile.

```
void bitonic_sort_v4(std::vector<int>& data) {
  // Create a SYCL queue
  sycl::queue q;
  // Create a buffer that wraps data
  sycl::buffer<int, 1> buf(data.data(),
    sycl::range<1>(N));
  // Set the tile size (work-group size)
  const size_t tile_size = 256;

  // Outer loop: size of the bitonic sequence
  for (size_t k = 2; k <= N; k *= 2) {
    // Inner loop: size of subsequences to
    compare and merge
    for (size_t j = k / 2; j > 0; j /= 2) {
      // Submit a command group to the SYCL
    queue
      q.submit([&](sycl::handler& h) {
        // Get access to the buffer
        auto d =
    buf.get_access<sycl::access::mode::read_write>(h);
        // Launch a parallel kernel
        h.parallel_for(
          sycl::nd_range<1>(sycl::range<1>(N),
    sycl::range<1>(tile_size)),
          [=](sycl::nd_item<1> item) {
            // Get the global ID of the current
    work item
            size_t i = item.get_global_id(0);
            // Compute the partner index using
    bitwise XOR
            size_t ixj = i ^ j;
            // Perform comparison and
    conditional swap
            if (ixj > i && ixj < N && i < N) {
              // Determine the sorting direction
              bool ascending = ((i & k) == 0);
              // Read the values to be compared
              int val_i = d[i];
              int val_ixj = d[ixj];
              // Swap values if they are out of
    order for the given direction
              if ((val_i > val_ixj) ==
    ascending) {
                d[i] = val_ixj;
                d[ixj] = val_i;
              }
            }
          }
        );
      });
    }
  }
  q.wait();
}
```

Listing 4. Bitonic Sort V4

All the versions while testing had a piece of code that was used for visualization and analysis of the intermediate steps:

```
Visualize the current state of the array after
    each step
q.submit([&](handler& h) {
  auto d =
    data_buf.get_access<access::mode::read>(h);
  h.host_task([=]() {
    std::ofstream outfile("bitonic_states.txt",
    std::ios::app);
    outfile << "k=" << k << ",j=" << j << ":";
    for (size_t idx = 0; idx < N; ++idx) {
        outfile << " " << d[idx];
    }
    outfile << "\n";
    });
});
```

Listing 5. Visualization

IV. Experimental Results

All the versions of the algorithm were tested in a same way using TiberAI[1] from Intel as a platform. Each algorithms was run with same sequence of data(100 different sets of data) in a serial order (data1 (v1,v2,v3,v4), data1(v1,v2,v3,v4) ....) to ensure that server availability does not impact relative results. Piece of code that was used for testing was:

```
void print_stats(const std::string& label,
    const std::vector<long long>& durations) {
  long long sum =
    std::accumulate(durations.begin(),
    durations.end(), 0LL);
  double average = static_cast<double>(sum) /
    durations.size();

  double variance = 0.0;
  for (auto d : durations) {
    variance += (d - average) * (d - average);
  }
  variance /= durations.size();
  double stddev = std::sqrt(variance);

  auto [min_it, max_it] =
    std::minmax_element(durations.begin(),
    durations.end());

  std::cout << label << " Stats
    (microseconds):\n";
  std::cout << "  Avg: " << average << "\n";
  std::cout << "  Std Dev: " << stddev << "\n";
  std::cout << "  Min: " << *min_it << "\n";
  std::cout << "  Max: " << *max_it << "\n";
  std::cout << std::endl;
}


int main() {
  constexpr int num_iterations = 100;
  constexpr int data_size = 1024;

  // Seed RNG for reproducibility (optional)
  std::srand(42);

  // Generate random base dataset ONCE
  std::vector<int> originalData(data_size);
  std::generate(originalData.begin(),
    originalData.end(), []() { return rand() %
    100000; });

  // Reference sorted result for validation
  std::vector<int> expected_data = originalData;
  std::sort(expected_data.begin(),
    expected_data.end());

  std::vector<int> currentData;
  std::vector<long long> durations;



  // -------- v1 --------
  durations.clear();
  for (int i = 0; i < num_iterations; ++i) {
    currentData = originalData;  // deep copy
    auto start =
    std::chrono::high_resolution_clock::now();
    bitonic_sort_v1(currentData);
    auto end =
    std::chrono::high_resolution_clock::now();

    if (i != 0)
    durations.push_back(

    std::chrono::duration_cast<std::chrono::microseconds>
      (end - start).count()
    );
    assert(currentData == expected_data &&
    "bitonic_sort_v1 failed.");
  }
  print_stats("Bitonic Sort v1", durations);

  // -------- v2 --------
  // Same as v1

  // -------- v3 --------
  // Same as v1

  // -------- v4 --------
  // Same as v1
```

Experimental setup.

Platform: Intel oneAPI (SYCL-compliant)

Dataset: 1024 randomly generated integers

Tools: SYCL queue execution on CPU or GPU (Results section was calculated using multiGPU TiberAI(Intel) setup, but development was done both on CPU and GPU.

Measurement: Execution time in microseconds, correctness checked via assertion.

Results.

Before first run of the first iteration, in order not to disturb the results, I needed to run one of the algorithms atleast once so that the data array gets into cache, before we do any time calculation. (that first iteration which has a lot of cache misses lasts 250 000 microseconds, while normal (all cache hits) iteration lasts 5000 microseconds)

Algorithm	Avg (μs)	Std Dev (μs)	Min (μs)	Max (μs)
Bitonic Sort v1	5287.32	255.50	2727	7707
Bitonic Sort v2	3039.09	135.06	2737	3688
Bitonic Sort v3	3571.83	320.45	2798	4587
Bitonic Sort v4	2566.12	323.80	2211	3535

TABLE I
PERFORMANCE STATISTICS FOR BITONIC SORT VARIANTS (IN MICROSECONDS).

Comments: The basic version is easy to understand but suffers from redundant global memory access.

The local memory version reduces this overhead, yielding moderate performance gains.

The subgroup version performs best when hardware supports efficient subgroup execution.

Tiling offers a balance between performance and implementation complexity, especially effective on memory-bound architectures.

V. CONCLUSIONS

This paper demonstrates that SYCL provides a viable platform for implementing portable and performant parallel sorting algorithms. Among the four tested versions of bitonic sort, local memory and tiling-based approaches achieved the best performance. Future work would include stacking multiple techniques under one algorithm, combining best parts of each technique to get an ultimate performance algorithm.

REFERENCES

[1] https://ai.cloud.intel.com/
[2] https://en.wikipedia.org/wiki/Bitonic_sorter